

mcp-ratchet — Tool Operational Requirements

The falsifiable "shall" statements the drift-detection function is qualified against: what a match is, what drift is, what must never be reported as "no drift", and the environment it holds in.

Doc ID: MR-TQ-002 Rev: 2026-08-29 Target TQL: 5 (per MR-TQ-001 §5)

Scope: deterministic drift-detection function only Read Order: 2 of 5

DO-330 asks that Tool Operational Requirements be clear, unambiguous, verifiable, and that they state what correspondence the tool's output must have to the activity it supports. These are written against the current mcp-ratchet source (`scanner/fingerprint.py` , `proxy/drift.py` , `proxy/run_proxy.py` , `proxy/server_side.py`). Where the tool does not yet meet a requirement as written, that is called out in §4 and carried to MR-TQ-004 — a TOR is a target, not a description.

§1 The activity being supported

From MR-TQ-001: a DO-178C program has qualified an AI assistant exposed as an MCP server, and needs ongoing assurance that the assistant's advertised tool surface has not changed since qualification. mcp-ratchet's drift-detection function provides that assurance. The correspondence its output must have to that activity:

Operational reliance claim — A "no drift" result for a server, evaluated against a stored baseline, shall mean that every tool the server currently advertises is canonically equivalent (TOR-2) to the tool of the same name recorded in that baseline, and that no advertised tool is absent from the baseline and no baselined tool is absent from the current surface.

§2 Operational environment

The requirements below are qualified only in this environment. A change to any row invalidates the qualification until re-verified.

ITEM	QUALIFIED CONFIGURATION
Host OS	Windows 10, 64-bit (the program's actual build/verification host). Not qualified on other platforms without re-verification.
Python	CPython 3.13.x. The fingerprint depends on <code>hashlib.sha256</code> , <code>json.dumps</code> ordering semantics, and dict key-sort behavior — all stable across 3.11–3.13, but the qualified point is 3.13.
mcp-ratchet version	The exact commit qualified in MR-TQ-005. <code>fingerprint_schema_version</code> must read <code>1</code> .
MCP SDK	<code>mcp >= 2.0.0</code> . The field set in TOR-1 is complete for the <code>Tool</code> model at 2.0.0; a future SDK that adds a model-visible field to <code>Tool</code> requires TOR-1 and re-verification.
Baseline file	<code>baselines/<slug>.json</code> , written once by <code>scanner.run_scan</code> at qualification time. Stored under program configuration management. Read-only to the operator; changes only through a controlled re-baseline with recorded authority.
Baseline integrity	The program is responsible for detecting unauthorized edits to the baseline file itself (checksum under CM, or equivalent). mcp-ratchet trusts the file it is given.

§3 Tool Operational Requirements

TOR-1 — surface capture

For each tool a server advertises, the baseline shall record the value of every field of the MCP `Tool` object that an MCP client can observe: at SDK 2.0.0 these are `name` , `title` , `description` , `input_schema` , `output_schema` , `annotations` , `icons` , and `meta` . A field that is absent or null in the advertised tool shall be recorded as absent, not as an explicit null.

TOR-2 — canonical equivalence ("match")

Two tool definitions shall be treated as matching if and only if their canonical forms are byte-identical, where the canonical form is produced by: recursively sorting all object keys; preserving array order; serialising with `json.dumps(sort_keys=True, separators=(",", ":"), ensure_ascii=True)` . Object-key order and insignificant JSON formatting shall not constitute a difference. Array order, and any change to a string value including whitespace, shall constitute a difference.

TOR-3 – added tool

If the current surface advertises a tool whose name is not in the baseline, the result shall be "drift", identifying that tool as added.

TOR-4 – removed tool

If the baseline records a tool whose name is not on the current surface, the result shall be "drift", identifying that tool as removed.

TOR-5 – modified tool

For a tool present in both the baseline and the current surface whose canonical forms are not equivalent per TOR-2, the result shall be "drift", identifying that tool and reporting which field or fields differ, with the baseline and current values of each.

TOR-6 – no cosmetic suppression

A difference that is whitespace-only after normalisation may be labelled as such for a human's benefit, but shall still be reported as drift. No label, flag, or heuristic shall cause a real field difference to be reported as "no drift".

TOR-7 – determinism

Given the same baseline file and the same advertised surface, the function shall produce the same result on every run, across process restarts and host reboots, independent of dict iteration order or the order in which the server lists its tools.

TOR-8 – missing baseline

If no baseline file exists for the target, the function shall report "baseline absent — drift not evaluated" and shall record that condition in the audit log. It shall not report "no drift", and its output shall not be usable as the assurance in §1.

TOR-9 – unreadable or malformed baseline

If a baseline file exists but cannot be parsed, is missing a required field, or declares a `fingerprint_schema_version` the running tool does not support, the function shall report an explicit baseline-error condition and record it in the audit log. It shall not report "no drift", and shall not crash the proxy such that tool calls proceed with no drift evaluation occurring silently.

TOR-10 – transparency

Detecting drift shall not alter what the client receives from the downstream server. The real `tools/list` result shall be forwarded unmodified; drift is reported alongside it, never by editing it.

TOR-11 – injection check carries no credit

The prompt-injection check (`scanner/checks/prompt_injection.py`) is outside the qualified function. Its output shall not be represented, in any report used for compliance credit, as a qualified result. Its presence or absence shall not affect the drift-detection result.

§4 Where the tool does not meet these yet

Honest gap list against the current source, to be closed or formally accepted in MR-TQ-004:

TOR	STATUS AGAINST CURRENT SOURCE
TOR-1–7	met <code>fingerprint.py</code> and <code>drift.py</code> implement this behaviour; <code>tests/test_fingerprint.py</code> and <code>test_drift.py</code> already exercise determinism, whitespace-still-trips, and add/remove/modify. MR-TQ-004 re-runs these as qualification evidence and adds the cases they miss.
TOR-8	partial A missing baseline prints a warning and logs a <code>no_baseline</code> record, and the session correctly does not claim "no drift". Not yet verified that a report generated in this state cannot be mistaken for a clean result by a downstream reader.
TOR-9	not met A malformed baseline raises an unhandled <code>json</code> or <code>KeyError</code> exception in <code>load_baseline</code> . There is no schema-version gate. This is the primary correctness gap the qualification effort has to close.
TOR-10	met <code>server_side.py</code> forwards the unmodified result; <code>tests/test_forward.py</code> asserts transparency.
TOR-11	partial The two checks are independent in code, but no report template yet carries an explicit "not qualified" marker on the injection section.

§5 The boundary that stays open even at TQL-5

TOR-1 captures every field of the `Tool` object. It does not capture everything an AI assistant acts on during an agentic loop. An MCP server also exposes resources, prompts, and server-level instructions, and it returns tool-call *results* — any of which can carry text that steers the model, and none of which are in the baseline.

STATED LIMIT OF THE QUALIFIED CLAIM

A qualified "no drift" result means the tool *surface* matches the baseline. It says nothing about resources, prompts, server instructions, or the content of tool-call results. The program needs separate controls for those. This study does not extend the fingerprint to cover them — doing so is a real, scoped piece of future work, not a claim made here.

WHY THE SURFACE IS STILL THE RIGHT THING TO QUALIFY FIRST

The tool surface is what a human reviews and approves when they decide to credit the assistant. It is the artifact the qualification is *about*. A control that holds exactly that artifact stable is the narrowest useful guarantee, and the one that fits a TQL-5 effort.

§6 Carried forward to MR-TQ-003

- Identify which DO-330 objectives apply at TQL-5 and how each is met for this tool. The exact objective list is table content in the standard and is not reproduced; MR-TQ-003 names each objective by intent and maps it to an artifact in this set.
- Plan the fix for TOR-9: a schema-version gate and explicit baseline-error handling in `load_baseline` , with the proxy failing safe rather than proceeding with silent non-evaluation.
- Plan the report-template markers for TOR-8 and TOR-11.
- Define configuration management for the baseline file and for the qualified mcp-ratchet commit itself.